

Z3-Z3++ in SMT-COMP 2026

Bohan Li^{1,2}, Shihao Jiao³, and Jun Wu³

¹ Institute of AI for industries, Chinese Academy of Sciences

² Jiangsu Key Laboratory of AI for Industries, Institute of AI for Industries, Chinese Academy of Sciences

³ Nanjing University of Information Science and Technology

bhli@iaii.ac.cn, 202512491182@nuist.edu.cn, wujun@nuist.edu.cn

Keywords: SMT solver · Derived solver · SMT COMP.

1 Introduction

Z3++ is a derived solver based on Z3 (version 4.8.7) [4]. Z3++ has participated in the SMT-COMP since 2022 [1]. An important feature of Z3++ is that it deeply integrates LocalSMT [2, 3, 6] as a new tactic. LocalSMT is a local search solver dedicated to SMT on arithmetic theories. This year, it participates in the single query track of QF_NIA and QF_DT. The usage is to run “./z3” command in the uploaded archive ⁴.

2 Technical Innovations

2.1 QF_NIA

Our implementation for the QF_NIA track is built on the 2023 version of Z3++ [1]. We used large language models (LLMs) to iteratively evolve the implementation and tune search heuristics.

The first improvement is the refinement of two-variable moves in the stochastic local search component, inspired by the MaxSMT local-search techniques of [5]. In the original design, clause repair and clause breakage were treated in a relatively symmetric manner, and candidate moves were mainly ranked by a single aggregated score. Our revised heuristic is more repair-oriented: repairing currently unsatisfied clauses is given higher priority than merely avoiding the destruction of satisfied clauses. This is especially helpful for nonlinear integer arithmetic, where progress often requires coordinated changes of two variables rather than conservative single-variable updates. Moreover, the move-selection strategy was iteratively tuned with the assistance of LLMs. The revised strategy combines two complementary scores to evaluate candidate two-variable moves, so that both their repair effect and their possible side effects are considered during selection.

The second improvement concerns the cooperation between stochastic local search and the CDCL(T)-style complete solver. We introduce an adaptive

⁴ <https://zenodo.org/records/20397586/files/z3upv1.zip>

threshold-based policy for partial SLS. After each SLS round, the solver checks the ratio of unsatisfied clauses. If this ratio is below a predefined threshold, the current state is considered promising, and the SLS budget for the next round is increased. Otherwise, the state is considered unpromising, the next SLS budget is decreased, and the complete solver is allowed to continue along a different search path. This strategy helps the solver concentrate local-search effort on near-satisfying assignments while preserving the robustness of the complete search engine.

2.2 QF_DT

Our participation in the QF_DT track focuses on a bounded SAT encoding for finite datatype fragments. The solver first performs a preprocessing step to detect datatype variables whose domains can be finitely bounded. This is straightforward for enumeration datatypes, and can also apply to datatypes constructed from enumeration types when preprocessing can determine a finite range of possible values.

For each bounded datatype variable, we enumerate a sufficiently large set of candidate values and introduce Boolean variables to encode the selected value. Constraints over datatype terms, such as equalities, disequalities, constructor recognizers, and selector constraints, are translated into propositional constraints based on their problem features. The translation is performed greedily: Boolean encodings are generated only as needed, and constraints are simplified immediately once enough information is available.

After this preprocessing phase, the QF_DT instance is reduced to a propositional SAT problem, which is solved by invoking the internal SAT solver. This encoding is designed to exploit the finite-domain structure of enumeration-based datatypes and to replace datatype-theory reasoning with efficient Boolean search whenever possible.

3 Website

<https://z3-plus-plus.github.io/>

4 Acknowledgments

We would like to thank the authors of Z3.

References

1. Cai, S., Li, B., Zhan, B., Zhang, X., Zhao, M.: Z3++ at smt-comp 2023
2. Cai, S., Li, B., Zhang, X.: Local search for smt on linear integer arithmetic. In: International Conference on Computer Aided Verification. pp. 227–248. Springer (2022)

3. Cai, S., Li, B., Zhang, X.: Local search for satisfiability modulo integer arithmetic theories. *ACM Transactions on Computational Logic* **24**(4), 1–26 (2023)
4. De Moura, L., Bjørner, N.: Z3: An efficient smt solver. In: *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. pp. 337–340. Springer (2008)
5. He, X., Li, B., Zhao, M., Cai, S.: A local search algorithm for maxsmt (lia). In: *International Symposium on Formal Methods*. pp. 55–72. Springer (2024)
6. Li, B., Cai, S.: Local search for smt on linear and multilinear real arithmetic. In: *FMCAD 2023*. pp. 168–177. TU Wien Academic Press (2023)